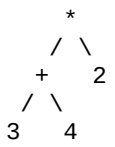
Recorrido inorden

Práctica: Árboles de expresiones · Identificador del problema: `inorden` · Límite de tiempo: 1 s

En un árbol de expresión las **hojas** son números y los **nodos internos** son operadores (`+ - * /`), cada uno con exactamente dos hijos. Como en la práctica, el recorrido **inorden** (izquierdo, raíz, derecho) da la notación **infija**, a la que se añaden paréntesis para que no sea ambigua.

Se te da el árbol ya construido. Tienes que escribirlo en notación infija implementando el método `infija()` del código base. Es el mismo método y el mismo formato que en la práctica, así que puedes reutilizar tu código.



Para (3 + 4) * 2, la infija es `((3 + 4) * 2)`.

Entrada

La primera línea contiene el número de casos `T` ($1 \le T \le 100$). Cada caso empieza con una línea con el número de nodos n ($1 \le n \le 1000$), seguida de n líneas, una por nodo, en el orden 0, 1, …, n – 1:

```
valor izquierdo derecho
```

- `valor` es un entero no negativo de como mucho 9 cifras o uno de los operadores `+ - * /`.
- `izquierdo` y `derecho` son los números de los nodos hijos, o `-1` si no hay hijo. **La raíz es siempre el nodo 0**; el resto de nodos puede aparecer en cualquier orden.
- El árbol siempre es correcto: los números son hojas y cada operador tiene dos hijos.

Salida

Para cada caso, una línea con la notación **infija** del árbol:

- Cada operación va entre paréntesis: `(izquierdo operador derecho)`.
- Los elementos van separados por un espacio y los paréntesis, pegados: `((3 + 4) * 2)`, no `( ( 3 + 4 ) * 2 )`.
- Una hoja va sin paréntesis. Si el árbol es solo un número, la salida es ese número: `7`.

No hay que evaluar la expresión (puede haber divisiones por cero).

Ejemplo

Ejemplo de entrada	Ejemplo de salida
3 5 * 1 2 + 3 4 2 -1 -1 3 -1 -1 4 -1 -1 1 7 -1 -1 5 - 2 1 - 3 4 8 -1 -1 4 -1 -1 1 -1 -1	((3 + 4) * 2) 7 (8 - (4 - 1))

Los casos son (3 + 4) * 2, el número 7 solo y 8 – (4 – 1). En el tercero, la raíz (nodo 0) tiene como hijo izquierdo el nodo 2 y como derecho el nodo 1.

Qué tienes que entregar

- Copia el código base de tu lenguaje (Python en esta página, C++ en la siguiente) en un fichero: `inorden.py` o `inorden.cpp`.
- Completa `infija()`. Puedes añadir los métodos auxiliares que necesites (el patrón público + auxiliar recursivo de `num_nodos`).
- No cambies la lectura ni la escritura y **no dejes ningún print de depuración**: cualquier texto de más da WRONG-ANSWER.
- Antes de enviarlo, pruébalo con el ejemplo. Por ejemplo: `python3 inorden.py < ejemplo.txt`.
- Si al copiar del PDF se pierde la sangría (en Python importa), descarga el fichero desde DomJudge o vuelve a sangrar con 4 espacios por nivel, sin tabuladores.

Código base en Python 3

```
# inorden.py - Codigo base para DomJudge (Arboles de expresiones)
# Completa infija(). Puedes anadir metodos auxiliares.
# No cambies la lectura ni la escritura, y no imprimas nada mas.
import sys

sys.setrecursionlimit(10000)


class Nodo:
    def __init__(self, valor, izquierdo=None, derecho=None):
        self.valor = valor
        self.izquierdo = izquierdo
        self.derecho = derecho

    def es_hoja(self):
        return self.izquierdo is None and self.derecho is None


class ArbolExpresion:
    def __init__(self):
        self.raiz = None

    def infija(self):
        # TODO: inorden.  (3 + 4) * 2  ->  "((3 + 4) * 2)"
        return ""


def leer_arbol():
    n = int(input())
    nodos = []
    hijos = []
    for i in range(n):
        valor, izq, der = input().split()
        nodos.append(Nodo(valor))
        hijos.append((int(izq), int(der)))
    for i in range(n):
        izq, der = hijos[i]
        if izq != -1:
            nodos[i].izquierdo = nodos[izq]
        if der != -1:
            nodos[i].derecho = nodos[der]
    arbol = ArbolExpresion()
    arbol.raiz = nodos[0]
    return arbol


casos = int(input())
for c in range(casos):
    arbol = leer_arbol()
    print(arbol.infija())
```

Código base en C++17

Copia dentro de la clase tu método de `arbol_expresiones.hpp`. Aquí todo va en un único `.cpp`, sin `#pragma once` ni otros ficheros. Si usas `std::` en tu código, funciona igual.

```
// inorden.cpp - Codigo base para DomJudge (Arboles de expresiones)
// Completa infija(). Puedes anadir metodos auxiliares.
// No cambies la lectura ni la escritura, y no imprimas nada mas.
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Nodo {
    string valor;
    Nodo* izquierdo = nullptr;
    Nodo* derecho = nullptr;

    bool es_hoja() const {
        return izquierdo == nullptr && derecho == nullptr;
    }
};

class ArbolExpresion {
public:
    Nodo* raiz = nullptr;

    string infija() const {
        // TODO: inorden.  (3 + 4) * 2  ->  "((3 + 4) * 2)"
        return "";
    }
};

int main() {
    int casos;
    cin >> casos;
    for (int c = 0; c < casos; c++) {
        int n;
        cin >> n;
        vector<Nodo> nodos(n);
        for (int i = 0; i < n; i++) {
            int izq, der;
            cin >> nodos[i].valor >> izq >> der;
            if (izq != -1) nodos[i].izquierdo = &nodos[izq];
            if (der != -1) nodos[i].derecho = &nodos[der];
        }
        ArbolExpresion arbol;
        arbol.raiz = &nodos[0];
        cout << arbol.infija() << "\n";
    }
    return 0;
}
```